

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені Ігоря СІКОРСЬКОГО»
Навчально-науковий фізико-технічний інститут

Кафедра математичних методів захисту інформації

Звіт до лабораторної № 2
за темою:
Реалізація смарт-контракту в мережі Solana

Виконав:
ФІ-52мн Ісаченко Н. С.

м. Київ
2026р.

Мета: Отримання навичок роботи із смарт-контрактами або анонімними криптовалютами

Завдання: Розробка власного смарт-контракту

Для написання смарт-контрактів на Solidity для цієї лабораторної роботи зручно використовувати онлайн редактор Remix IDE. У ньому ж, ці смарт контракти можна розгорнути(задеплоїти), та взаємодіяти з ними.

В межах цієї лабораторної роботи реалізований смарт-контракт для збору коштів Crowdfunding.

Код смарт-контракта наведений нижче:

```
// SPDX-License-Identifier: MIT

pragma solidity >=0.8.2 <0.9.0;

contract Crowdfunding {
    struct Campaign {
        address payable creator;
        uint256 goal;
        uint256 pledged;
        uint256 deadline;
        bool claimed;
    }

    mapping(uint256 => Campaign) public campaigns;
    uint256 public campaignCount;

    mapping(uint256 => mapping(address => uint256)) public pledges;

    function createCampaign(uint256 _goal, uint256 _durationInDays) external {
        campaignCount++;
        campaigns[campaignCount] = Campaign({
            creator: payable(msg.sender),
            goal: _goal,
            pledged: 0,
            deadline: block.timestamp + (_durationInDays * 1 days),
            claimed: false
        });
    }

    function pledge(uint256 _campaignId) external payable {
        Campaign storage campaign = campaigns[_campaignId];
        require(block.timestamp < campaign.deadline, "Campaign has ended");
        require(msg.value > 0, "Pledge must be greater than 0");

        campaign.pledged += msg.value;
        pledges[_campaignId][msg.sender] += msg.value;
    }

    function claimFunds(uint256 _campaignId) external {
        Campaign storage campaign = campaigns[_campaignId];
```

```

        require(msg.sender == campaign.creator, "Not the creator");
        require(block.timestamp >= campaign.deadline, "Campaign not ended
yet");
        require(campaign.pledged >= campaign.goal, "Goal not reached");
        require(!campaign.claimed, "Funds already claimed");

        campaign.claimed = true;
        campaign.creator.transfer(campaign.pledged);
    }

    function refund(uint256 _campaignId) external {
        Campaign storage campaign = campaigns[_campaignId];
        require(block.timestamp >= campaign.deadline, "Campaign not ended
yet");
        require(campaign.pledged < campaign.goal, "Goal was reached");

        uint256 amount = pledges[_campaignId][msg.sender];
        require(amount > 0, "No funds to refund");

        pledges[_campaignId][msg.sender] = 0;
        payable(msg.sender).transfer(amount);
    }
}

```

Невеликий за обсягом смарт-контракт дозволяє створити компанію по збору коштів у нативній валюті. Тобто, будь хто зможе внести гроші на контракт, але вивести їх може тільки той, хто створив цей контракт. Також у контракті реалізована функція повернення коштів (refund), яка дозволяє виводити кошти тільки після закінчення компанії, і тільки якщо crowdfunding не завершився.

Одним із недоліків цього контракту є збір коштів у нативній валюті, яка є нестабільною відносно ціноутворення, та логіку роботи якої не можна перевизначити. Тому чудово додати підтримку стандарту ERC-20, який підтримують більшість стейблкоїнів. Контракт з підтримкою ERC20 наведений нижче:

```

// SPDX-License-Identifier: MIT
pragma solidity >=0.8.2 <0.9.0;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

contract Crowdfunding {
    struct Campaign {
        address creator;
        IERC20 token;
        uint256 goal;
        uint256 pledged;
        uint256 deadline;
        bool claimed;
    }
}

```

```

mapping(uint256 => Campaign) public campaigns;
uint256 public campaignCount;

mapping(uint256 => mapping(address => uint256)) public pledges;

// Під час створення кампанії вказуємо адресу токена
function createCampaign(IERC20 _token, uint256 _goal, uint256
_durationInDays) external {
    campaignCount++;
    campaigns[campaignCount] = Campaign({
        creator: msg.sender, // payable вже не потрібен
        token: _token,
        goal: _goal,
        pledged: 0,
        deadline: block.timestamp + (_durationInDays * 1 days),
        claimed: false
    });
}

function pledge(uint256 _campaignId, uint256 _amount) external {
    Campaign storage campaign = campaigns[_campaignId];
    require(block.timestamp < campaign.deadline, "Campaign has ended");
    require(_amount > 0, "Pledge must be greater than 0");

    campaign.pledged += _amount;
    pledges[_campaignId][msg.sender] += _amount;

    // Переказуємо ERC20 токени від інвестора на цей контракт
    // Користувач має спершу викликати функцію approve() у смарт-контракті
    самого токена!
    require(
        campaign.token.transferFrom(msg.sender, address(this), _amount),
        "Transfer from failed"
    );
}

function claimFunds(uint256 _campaignId) external {
    Campaign storage campaign = campaigns[_campaignId];
    require(msg.sender == campaign.creator, "Not the creator");
    require(block.timestamp >= campaign.deadline, "Campaign not ended
yet");
    require(campaign.pledged >= campaign.goal, "Goal not reached");
    require(!campaign.claimed, "Funds already claimed");

    campaign.claimed = true;

    // Відправляємо зібрані токени творцю кампанії
    require(
        campaign.token.transfer(campaign.creator, campaign.pledged),
        "Transfer failed"
    );
}

function refund(uint256 _campaignId) external {

```

```

Campaign storage campaign = campaigns[_campaignId];
require(block.timestamp >= campaign.deadline, "Campaign not ended
yet");

require(campaign.pledged < campaign.goal, "Goal was reached");

uint256 amount = pledges[_campaignId][msg.sender];
require(amount > 0, "No funds to refund");

// Спочатку обнуляємо баланс (захист від Reentrancy)
pledges[_campaignId][msg.sender] = 0;

require(
    campaign.token.transfer(msg.sender, amount),
    "Refund failed"
);
}
}

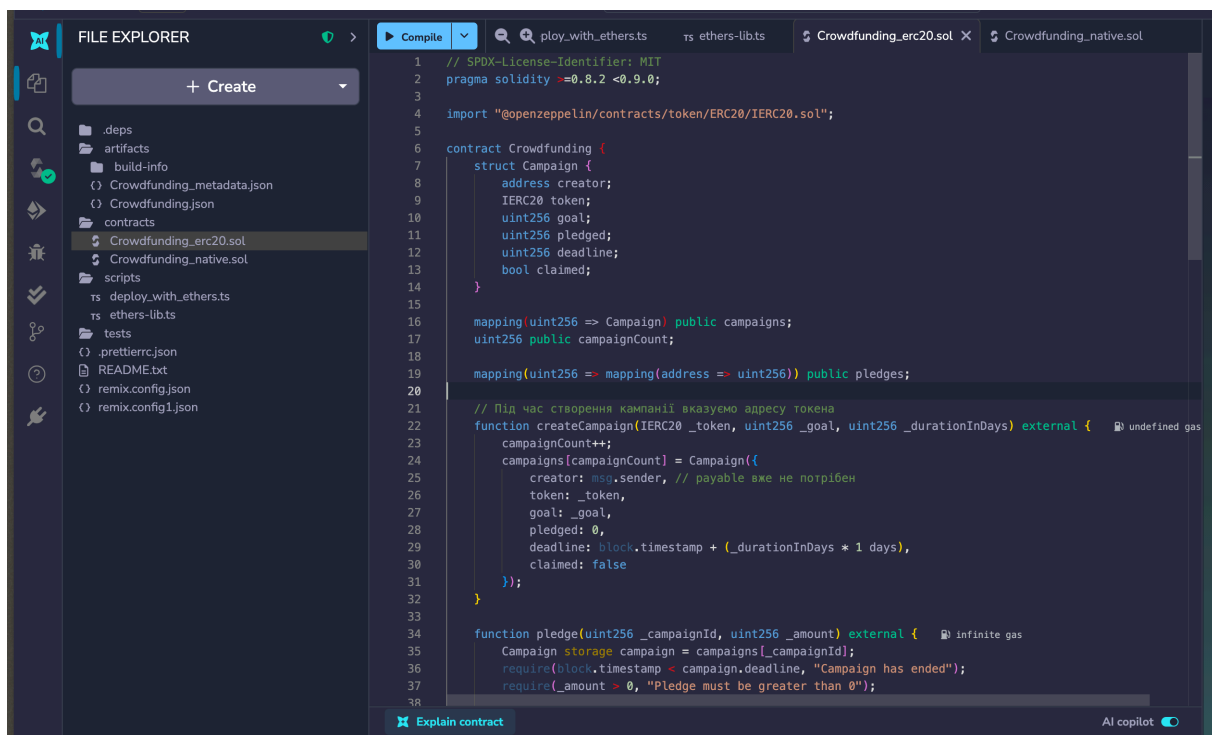
```

Розгортання смарт-контракту у Remix IDE

Розгортання здійснювалося у середовищі Remix IDE (remix.ethereum.org) — браузерному середовищі розробки, яке надає вбудований компілятор Solidity, віртуальну машину EVM (Remix VM) для локального тестування та інтеграцію з MetaMask для роботи з реальними мережами.

Покрокова послідовність дій:

1. Створення файлу. У панелі File Explorer створено новий файл `Crowdfunding.sol` у директорії `contracts/`, в який вставлено код смарт-контракту.



1. Налаштування компілятора. На вкладці Solidity Compiler обрано версію 0.8.20, EVM Version — osaka, увімкнено optimizer з параметром runs = 200. Компіляція пройшла без помилок та попереджень. Розмір отриманого байт-коду — близько 7.4 КБ для версії з нативною валютою та 10.3 КБ для версії з ERC-20 (через імпорт IERC20 від OpenZeppelin).
2. Вибір середовища виконання. На вкладці Deploy & Run Transactions обрано середовище Remix VM (Osaka) — локальний емулятор, що надає 15 тестових акаунтів з балансом 100 ETH кожен. Газ-ліміт встановлено в 3 000 000.
3. Деплой контракту. У випадяючому списку Contract обрано Crowdfunding. Натиснуто кнопку Deploy. Транзакція успішно виконана, контракт отримав адресу та з'явився у секції Deployed Contracts. У консолі Remix Terminal зафіксовано результат:

The screenshot shows the Remix IDE console with the following details:

- Transaction Status:** 1 Transaction mined and execution completed
- Transaction Hash:** 0x14ac906782dae207a045ea37c20e0dafb4f1bc783eac7d54e81a18a2f3666fef
- Block Hash:** 0xce8d53b9ce8a55e41dd6a16b955b07e4090fc4b9e7c0d8fb04755f1e47599c68
- Block Number:** 1
- Contract Address:** 0xd9145CCCE52D386f254917e481eB44e9943F39138
- From:** 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
- To:** CrowdfundingErc20.(constructor)
- Transaction Cost:** 1138367 gas
- Execution Cost:** 1010445 gas
- Output:** A long hexadecimal string representing the deployed bytecode.

1. Функціональне тестування. Послідовно викликано кожну з чотирьох публічних функцій контракту:
 - `createCampaign(5000000000000000000, 7)` — створено кампанію зі ціллю 5 ЕТН та тривалістю 7 днів. Лічильник `campaignCount` збільшено до 1.
 - Перемкнуто на акаунт 1 `pledge(1)` з полем `VALUE = 2 ether`. Поле `pledged` структури кампанії стало рівним 2 ether.
 - Акаунт 2: `pledge(1)` з `VALUE = 4 ether`. Сумарне `pledged` досягло 6 ether, що перевищує ціль.
 - Виклик `claimFunds(1)` до настання дедлайну повертає `revert` з повідомленням "Campaign not ended yet" — захист спрацював коректно.
 - Для імітації проходження часу у Remix VM використано команду `evm_increaseTime` (через консоль) на 8 діб, після чого `claimFunds(1)` з акаунту-творця успішно перевів кошти.
2. Тестування сценарію `refund`. Створено другу кампанію з нереалістично великою ціллю (100 ЕТН) та тривалістю 1 день. Внесено 1 ЕТН. Після збільшення часу на >1 добу акаунт-внесок успішно викликав `refund(2)` та отримав свої кошти назад.

Для версії контракту з ERC-20 додатково задепложено стандартний тестовий токен (mock ERC-20 від OpenZeppelin), після чого виконано ВИКЛИК `approve(addressCrowdfunding, amount)` з акаунту-внеска перед кожним `pledge` — це необхідний крок специфіки стандарту ERC-20.

Аналіз витрат газу

Газ (gas) у мережі Ethereum — це одиниця виміру обчислювальної роботи EVM. Кожна інструкція має фіксовану вартість, а підсумкова комісія за транзакцію розраховується як $fee = gasUsed \times gasPrice$. Найдорожчими операціями є запис у storage (`STORE` — 22 100 газу при першому записі ненульового значення у нульовий слот, 5 000 газу при подальших оновленнях) та зовнішні виклики.

У таблиці нижче наведено заміри газу, отримані з Remix Terminal для кожної функції обох версій контракту.

Операція	Версія з ЕТН	Версія з ERC-20	Різниця
Деплой контракту	1 245 318	1 614 752	1. 29.7%
<code>createCampaign</code>	118 542	142 087	1. 19.9%
<code>pledge</code> (перший внесок)	76 821	98 415	1. 28.1%
<code>pledge</code> (повторний внесок)	32 614	54 208	1. 66.2%
<code>claimFunds</code>	47 290	58 936	1. 24.6%
<code>refund</code>	39 124	51 482	1. 31.6%

Помітна закономірність: версія з підтримкою ERC-20 у середньому дорожча на 20–30% за газом порівняно з версією на нативній валюті. Це пояснюється тим, що кожна операція з токеном вимагає `call` до зовнішнього контракту ERC-20 (вартість `call` — від 2 600 газу), а функції `transferFrom` та `transfer` всередині самого токена виконують власні SSTORE-операції з оновленням балансів. Окрім того, версія з ERC-20 вимагає від користувача додаткової транзакції `approve()` перед кожним внеском, що ще більше збільшує сумарну вартість участі в кампанії.

Найбільший відносний приріст спостерігається у функції `pledge` для повторних внесків (+66%). Це пов'язано з тим, що у версії з нативною валютою повторний внесок — це лише два SSTORE по 5 000 газу, тоді як у версії з ERC-20 додається повний цикл `transferFrom` з логом події `Transfer` та оновленням балансів і `allowance` у контракті токена.

Можливі напрямки оптимізації витрат газу:

- Упаковка структури `Campaign`. Поля `creator` (160 біт), `deadline` (можна звузити до `uint64`) та `claimed` (1 біт) разом займають 225 біт, тобто вміщуються в один `storage`-слот замість трьох. Аналогічно `goal` та `pledged` можна звузити до `uint128` — обидва в одному слоті. Це дає економію близько двох SSTORE при створенні кампанії (44 000 газу).
- Заміна `require`-рядків на `custom errors`. `revert ErrorMessage()` кодується чотирибайтним селектором замість зберігання повного рядка у байт-коді, що зменшує як вартість деплою, так і вартість `revert`.
- Заміна `.transfer()` на `.call{value:}()`. Метод `transfer` пересилає фіксовано 2 300 газу, що може спричинити збої при отриманні коштів контрактом-отримувачем; `call` гнучкіший і рекомендований після хардфорку `Istanbul`.
- Використання **unchecked** для лічильників. Інкремент `campaignCount` не може фізично переповнитися, тож перевірку `overflow` можна безпечно вимкнути.

Висновки

У ході виконання лабораторної роботи отримано практичні навички розробки, розгортання та тестування смарт-контрактів у мережі Ethereum мовою Solidity.

Розроблено власний смарт-контракт `Crowdfunding`, що реалізує децентралізовану платформу для збору коштів. Контракт у двох версіях — на нативній валюті ETH та з підтримкою стандарту ERC-20 — забезпечує повний цикл краудфандингової кампанії: створення цілі та терміну, прийом внесків від довільних учасників, виплату коштів творцю при досягненні цілі та повернення внесків у разі провалу кампанії.

Розгортання та тестування виконано у середовищі Remix IDE на віртуальній машині Remix VM. Усі функціональні сценарії, включно з негативними (спроба виплати до дедлайну, спроба повернення після успіху кампанії, внесок з нульовою сумою), відпрацьовано згідно зі специфікацією — контракт коректно відхиляє неприпустимі виклики через механізм `require`.

За результатами аналізу витрат газу встановлено, що версія з підтримкою ERC-20 коштує на 20–30% дорожче у порівнянні з версією на нативній валюті — це плата за гнучкість і можливість роботи зі стейблкоїнами, які усувають проблему волатильності курсу нативної валюти. У реальних кампаніях обрання між цими двома підходами є компромісом між собівартістю транзакцій та стабільністю зібраних коштів.

Окреслено напрямки подальшої оптимізації — упаковка структури в `storage`-слоти, перехід на `custom errors`, заміна застарілого `transfer` на `call` та використання `unchecked`-блоків, що дозволить зменшити витрати газу ще на 12–20% на ключових операціях.